



Adrián Arsenio Garcés Jimenez

DESARROLLADOR BACKEND · JAVA & SPRING
BOOT

PERFIL PROFESIONAL

Desarrollador Backend autodidacta especializado en Java 21 y Spring Boot. Diseño y publico software en producción de principio a fin: una librería Open Source en Maven Central y sistemas distribuidos con microservicios y Apache Kafka, aplicando patrones SAGA y Circuit Breaker para garantizar consistencia e idempotencia bajo fallos. Mi enfoque es orientado a resultados medibles y a completar el ciclo completo de desarrollo — de arquitectura a despliegue — no solo a conocer tecnologías de forma aislada.

HABILIDADES

Lenguajes & Frameworks	Java 21+, Spring Boot, Spring Security, Spring Data JPA, Spring Cloud, NestJS (TypeScript), React
Arquitectura	Microservicios, Apache Kafka (event-driven), Clean Architecture, patrón SAGA (coreografía), Circuit Breaker (Resilience4J)
Seguridad	Diseño de esquemas completos de autenticación, autorización y auditoría; OAuth2 (Authorization Code + token relay), estándares SAML/OAuth2, gestión de identidad (IAM)
Datos	PostgreSQL, MySQL, modelado de dominio con JPA
Herramientas	Docker, Docker Compose, Git/GitHub, documentación de API con OpenAPI, CI/CD
Habilidades Blandas	Colaboración open source (mantenimiento de librería con colaboradores externos), resolución de problemas distribuidos, aprendizaje autónomo demostrado (Java, Spring, Kafka y patrones cloud-native en paralelo a estudios a tiempo completo)

PROYECTOS DESTACADOS

Prompt Link

JAVA 21 · SPRING BOOT 4 · MAVEN CENTRAL

Cliente Java reutilizable para IA generativa (OpenRouter: GPT-4, Claude, Llama) con autoconfiguración Spring Boot, publicado en Maven Central bajo licencia MIT.

- Diseñé el interceptor de autenticación para que viva únicamente en el contexto del Feign Client de OpenRouter, evitando que la API key se filtrara hacia otros Feign clients de la aplicación consumidora.
- Implementé manejo de errores tipado (AiClientException) que distingue error HTTP, fallo de red, respuesta inválida y error de configuración mediante códigos de estado propios (-1 a -4).
- Añadí streaming de tokens sobre WebFlux (Flux<String>) y reintentos configurables con backoff exponencial; publiqué la versión 1.1.0 con 22 commits y 1 fork externo.
- Documenté en el changelog un fix de seguridad crítico (fuga de API key entre Feign clients) detectado y corregido en la versión 1.0.3.

FinanceTracker Monolith-API

JAVA 21 · SPRING BOOT · POSTGRESQL · IA

API de finanzas personales que integra **Prompt Link** (mi propia librería) para análisis automático de gastos, categorización y un asistente financiero conversacional.

- Modelé el dominio Usuario → Cuenta → Transacción con aislamiento estricto de datos y generé reportes financieros dinámicos por mes/año sobre una arquitectura en capas (controller, service, repository, entity, dto, mapper, security, exception, handler).
- Aseguré la API con JWT y Spring Security, y centralicé el manejo global de errores en una capa handler/ dedicada con @RestControllerAdvice.
- Expuse 3 endpoints de IA (análisis automático, chat financiero, categorización de gastos) consumiendo mi propia librería Prompt Link en producción, validándola en un caso de uso real.

OrderFlow

MICROSERVICIOS · KAFKA · OAUTH2 · DOCKER

Plataforma de pedidos con 8 microservicios (Java 21, Spring Boot 4, Spring Cloud), eventos asíncronos con Kafka y seguridad OAuth2 de extremo a extremo.

- Diseñé el flujo de pedidos con patrón **SAGA por coreografía**: Order → reserva de stock en Inventory → Payment → confirmación o rollback, coordinado íntegramente vía eventos Kafka entre servicios.
- Implementé un Authorization Server propio (OAuth2, flujo Authorization Code) con el Gateway como cliente OAuth2 que valida y propaga el JWT a los microservicios internos (token relay).
- Añadí Circuit Breaker (Resilience4J) en los Feign clients de product e inventory, e idempotencia por pedido mediante una clave UUID para evitar procesamiento duplicado.
- Integré notificaciones por email (MailHog), documenté la API con OpenAPI/Swagger en el Gateway y orquesté los 8 servicios (discovery, gateway, auth, user, product, inventory, order, payment, notification) con Docker Compose (Kafka + Zookeeper).

Más proyectos (incluye **ChallengeHub**: API de gamificación con JWT y control de acceso por roles) disponibles en adrian0511.dev y github.com/adrian0511

EDUCACIÓN

Universidad de Las Tunas	2024 — 2026
Ingeniería Informática · 2do año	
IPVCE Luis Urquiza Jorge	2020 — 2023
Bachillerato	

IDIOMAS

Español	Nativo
Inglés	A2